# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1702**

---

**COURSE OUTCOME COVERED**

***CO1:*** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5** **Duration : 1 Hour**

**Total Marks:50**

Annexure A

## Analytical Problem Solving

---

***Code of Conduct:***

*I SAI GOPAL REDDY . Y(192524083) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

***SAI GOPAL REDDY***

**QUESTION 1**

Solve the Water Jug problem: You are given two jugs, a 4-gallon jug and a 3-gallon jug. Neither jug has measuring marks. A pump can be used to fill the jugs. How can exactly 2 gallons of water be obtained in the 4-gallon jug? Assume that a jug may be filled from the pump, emptied onto the ground, or poured into the other jug, and that no other measuring device is available.

**Problem understanding and representation**

Let a state be represented as (x, y), where x is the amount of water in the 4-gallon jug and y is the amount of water in the 3-gallon jug. Therefore, the capacity constraints are 0 â‰¤ x â‰¤ 4 and 0 â‰¤ y â‰¤ 3.

Initial state: (0, 0)

Goal state: (2, y), where 0 â‰¤ y â‰¤ 3; the required condition is specifically x = 2.

Permitted operators: Fill either jug, empty either jug, or pour water from one jug into the other until the source is empty or the destination is full.

**Systematic solution**

| Step | Operation | 4-gallon jug (x) | 3-gallon jug (y) | State / explanation |
|---|---|---|---|---|
| 0 | Initial state | 0 | 0 | (0, 0) |
| 1 | Fill the 4-gallon jug from the pump | 4 | 0 | (4, 0) |
| 2 | Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full | 1 | 3 | (1, 3) |
| 3 | Empty the 3-gallon jug onto the ground | 1 | 0 | (1, 0) |
| 4 | Pour the remaining 1 gallon from the 4-gallon jug into the 3-gallon jug | 0 | 1 | (0, 1) |
| 5 | Fill the 4-gallon jug from the pump | 4 | 1 | (4, 1) |
| 6 | Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full | 2 | 3 | (2, 3) |

**Final answer: The 4-gallon jug contains exactly 2 gallons. The solution requires six legal operations from the initial state, and no measuring marks or external measuring device are used.**

**Verification**

At Step 5, the 4-gallon jug contains 4 gallons and the 3-gallon jug contains 1 gallon. The 3-gallon jug has room for 2 more gallons. Hence exactly 2 gallons are transferred from the 4-

gallon jug, leaving 4 âˆ’ 2 = 2 gallons in the 4-gallon jug. The result satisfies the goal condition.

**QUESTION 2**

Find out how a Mars Rover can analyse and explore samples on Mars by answering the following questions.

(a) What are the percepts for this agent?

A percept is any information received by the rover through its sensors. The Mars rover must combine sensory information about its surroundings, its internal condition, scientific samples, and communication status.

Percept category Examples of percepts Purpose

Terrain and obstacles Stereo-camera images, rocks, slopes, cliffs, sand, dust, trenches and terrain roughness Safe navigation and path planning

Position and motion Inertial measurements, wheel rotation, visual odometry, celestial references and estimated coordinates Localization and movement control

Scientific observations Rock and soil images, mineral spectra, chemical composition, temperature, radiation and atmospheric readings Identify scientifically valuable targets

Sample information Sample location, hardness, texture, mass, drilling resistance and contamination indicators Decide whether and how to collect and analyse a sample

Internal health Battery level, solar energy, temperature, actuator condition, memory and instrument status Prevent damage and manage limited resources

Communication Earth-command messages, acknowledgement status, signal strength and communication-window availability Receive objectives and transmit findings

(b) Characterize the operating environment

The rover operates in a partially observable, stochastic, sequential, dynamic, continuous, unknown and multi-agent environment.

Property Characterization and justification

Partially observable The rover cannot directly observe the entire landscape, subsurface structure, future weather or all hazards. It must maintain an internal estimate of the world.

Stochastic Wheel slippage, dust storms, mechanical uncertainty, sensor noise and unexpected terrain make outcomes uncertain.

Sequential Each action changes later possibilities: a route choice affects energy, position, safety and future scientific access.

Dynamic Lighting, temperature, wind, dust and the rover's own battery state change while it is operating.

Continuous Position, time, speed, energy, temperature and terrain slope are naturally continuous variables.

Unknown Before exploration, the rover does not possess a complete map or complete knowledge of sample locations.

Multi-agent The rover interacts with human controllers and communication systems; other autonomous systems may also influence mission planning.

(c) What actions can the agent take?

Move forward, reverse, turn, rotate in place and select a safe path.

Stop, reduce speed, avoid an obstacle and recover from wheel slip.

Deploy or position the robotic arm and select a target location.

Drill, scoop, collect, transfer and store rock or soil samples.

Perform imaging, spectroscopy, chemical analysis and environmental measurements.

Transmit images, measurements, health data and scientific results to Earth.

Receive and execute mission commands, recharge or conserve energy, enter safe mode, and schedule operations around communication windows.

(d) How can the performance of the agent be evaluated?

Performance should be measured using a utility function that balances scientific benefit, safety, mission progress and resource usage. A representative performance measure is:

U = ws(scientific value) + wp(mission progress) + wd(data quality) â^’ wr(risk) â^’ we(energy consumption) â^’ wt(time) â^’ wf(fault cost)

Important measurable indicators include:

Number and scientific importance of samples correctly identified, collected and analysed.

Accuracy, completeness and transmission success of scientific data.

Distance or number of mission objectives completed per unit of energy and time.

Navigation safety: absence of collisions, immobilization and irreversible damage.

Reliability of autonomous decisions and percentage of operations completed without intervention.

Effective use of communication windows, storage capacity, instruments and battery power.

Ability to recover from uncertainty, faults and changing environmental conditions.

(e) Most suitable agent architecture and justification

The most suitable architecture is a model-based, utility-based, goal-oriented learning agent, implemented as a hybrid architecture.

Component Role in the Mars rover

Perception module Converts camera, spectrometer, navigation and health-sensor readings into useful percepts.

Internal world model Maintains a local map, rover position, terrain hazards, sample beliefs, battery state and instrument condition despite partial observability.

Goal manager Represents goals such as reach a target, inspect a sample, return to a safe point or transmit data.

Utility-based planner Ranks alternative plans by scientific value, safety, energy, time, risk and communication constraints.

Learning and adaptation module Improves terrain classification, slip prediction, sample selection and recovery policies from experience.

Execution and safety layer Converts plans to actuator commands and overrides unsafe actions with emergency stops or safe mode.

A simple reflex agent is insufficient because the rover cannot act correctly from only the current percept; it must remember past observations. A purely goal-based agent is also insufficient because several plans may reach the same goal with different scientific value and risk. A utility-based, model-based architecture is therefore preferred, while learning allows performance to improve over time.

**QUESTION 3**

Explore the search space using search strategies and formulate the problem components for the 8-Queens problem: Place 8 queens on a chessboard so that no queen attacks any other queen.

**Problem formulation**

Component Formal definition

State A partial or complete placement of queens. A convenient representation is [r1, r2, â€¦, rk], where ri is the row occupied by the queen in column i.

Initial state [] â€” an empty board with no queens placed.

Actions Place one queen in the next unoccupied column in any row that is not attacked by an already placed queen.

Transition model Appending a legal row r to the current state produces [r1, â€¦, rk, r].

Goal test Eight queens have been placed and for every pair i â‰  j: ri â‰  rj and |ri â€’ rj| â‰  |i â€’ j|.

Path cost Uniform cost of 1 per queen placement. Every complete solution has cost 8; therefore the main objective is to find any valid goal state.

Constraint No two queens may share a row, column or diagonal. The column restriction is automatically satisfied by placing one queen per column.

**Search-space analysis**

If one queen is placed in each column and every row is initially considered, the unpruned search space has 88 = 16,777,216 possible row sequences. Constraint checking prunes illegal partial states immediately. If row uniqueness is also enforced before diagonal checking, the remaining arrangements are at most 8! = 40,320, after which diagonal constraints reduce the space further. There are 92 ordered solutions, or 12 fundamental solutions when rotations and reflections are treated as equivalent.

**Search strategies**

Strategy Application to 8-Queens Evaluation

Breadth-First Search (BFS) Expands all legal placements at depth 0, then depth 1, and so on. Complete, but stores a large frontier and is unnecessary because all solutions have the same depth.

Depth-First Search (DFS) Places queens column by column and follows one branch until it fails or reaches a goal. Low memory and natural for this problem; with backtracking it is efficient and complete for the finite tree.

Depth-Limited Search DFS with a limit of 8 placements. Suitable because a solution must have exactly eight levels.

Backtracking with forward checking Rejects a row as soon as it conflicts and maintains the remaining legal rows. Most practical formulation; it prevents expansion of impossible partial configurations.

**Backtracking algorithm**

```
PROCEDURE Solve8Queens(board, column):

IF column = 9 THEN

RETURN SUCCESS                // eight queens placed

FOR row â†□ 1 TO 8 DO

IF IsSafe(board, row, column) THEN

board[column] â†□ row

IF Solve8Queens(board, column + 1) = SUCCESS THEN

RETURN SUCCESS

board[column] â†□ EMPTY       // backtrack after failure

RETURN FAILURE
```

```
FUNCTION IsSafe(board, row, column):
FOR previousColumn â†□ 1 TO column â^' 1 DO
previousRow â†□ board[previousColumn]
IF previousRow = row THEN
RETURN FALSE            // same row
IF |previousRow â^' row| = |previousColumn â^' column| THEN
RETURN FALSE            // same diagonal
RETURN TRUE
```

**One valid solution and verification**

One solution, represented by rows for columns 1 through 8, is:

[1, 5, 8, 6, 3, 7, 2, 4]

| Column | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Row | 1 | 5 | 8 | 6 | 3 | 7 | 2 | 4 |

All row values are distinct, so no two queens share a row. Since exactly one queen is placed in each column, no two share a column. Checking every pair also confirms that the absolute row difference never equals the absolute column difference; therefore no pair lies on the same diagonal. This configuration is a valid goal state.

**QUESTION 4**

A customer wants to travel from one location to another using the OLA Cab booking mobile application. The customer can select a cab type such as mini, micro, sedan, shared, prime and so on based on comfort to reach the destination. Identify the type of problem-solving agent and write the pseudocode.

**Problem understanding and agent selection**

The task is to select and book a feasible cab that takes the customer from the source to the destination while satisfying preferences such as cab type, comfort, price, waiting time and arrival time. The most appropriate agent is a goal-based agent enhanced with utility-based decision-making.

The goal is to reach the destination safely and complete a booking.

A goal alone may permit several cabs; therefore a utility function ranks feasible alternatives by comfort, fare, ETA, travel time, safety and customer preference.

Because traffic and cab availability change, the agent must use current percepts and re-plan when a selected cab becomes unavailable.

The agent is therefore also model-based in practice, because it maintains the current booking state and trip information.

**Problem formulation**

Component Definition

Initial state Customer is at source location S, has destination G, and has stated preferences.

Actions Enter locations, select cab type, request fare estimates, compare cabs, confirm booking, cancel or reselect, and track the trip.

State information GPS position, destination, available cabs, type, fare, ETA, route, comfort rating, traffic and booking status.

Goal test A suitable cab is successfully booked and the customer reaches G safely.

Utility Higher comfort and safety, lower fare, shorter waiting/travel time, and better match with the customer's preference.

**Utility model**

Utility(cab) = wcComfort + wsSafety â^’ wfFare â^’ wwWaitingTime â^’ wtTravelTime

The weights are set according to the customer's stated priorities. For example, if comfort is most important, wc is increased; if the customer wants the lowest price, wf is increased.

**Pseudocode**

PROCEDURE BookOLACab(source, destination, preferences):

REQUIRE source â‰  destination

IF source or destination is invalid THEN

DISPLAY "Enter valid pickup and destination locations"

RETURN FAILURE

routeInfo â† GetRouteEstimate(source, destination)

availableCabs â† GetAvailableCabs(source)

IF availableCabs is EMPTY THEN

DISPLAY "No cabs are currently available"

OFFER retry or notify-me option

RETURN FAILURE

feasibleCabs â† EMPTY_LIST

FOR each cab IN availableCabs DO

IF cab.type satisfies preferences.allowedTypes AND

cab.seats â‰¥ preferences.requiredSeats AND

cab.isSafe = TRUE THEN

estimate â† EstimateFareETA(cab, routeInfo, preferences)

utility â† CalculateUtility(estimate, cab, preferences)

ADD (cab, estimate, utility) TO feasibleCabs

IF feasibleCabs is EMPTY THEN

DISPLAY "No cab satisfies the selected requirements"

OFFER alternative cab types or updated preferences

RETURN FAILURE

SORT feasibleCabs BY utility in descending order

selected â† feasibleCabs[0]

DISPLAY selected cab type, fare, ETA, route and comfort details

IF CustomerConfirms(selected) = FALSE THEN

selected â† CustomerSelects(feasibleCabs)

IF selected = NONE THEN

RETURN CANCELLED

booking â† RequestBooking(selected.cab, source, destination)

IF booking.status = CONFIRMED THEN

DISPLAY booking ID, driver details, pickup ETA and fare estimate

WHILE trip is not completed DO

location â† TrackCabAndCustomer()

IF route hazard or cab failure is detected THEN

RecalculateRouteOrOfferReplacement()

DISPLAY "Trip completed successfully"

RETURN SUCCESS

ELSE

REMOVE selected FROM feasibleCabs

IF feasibleCabs is not EMPTY THEN

REPEAT booking with the highest-utility remaining cab

ELSE

DISPLAY "Booking failed; please try again"

RETURN FAILURE

**Interpretation**

This agent does not blindly choose the cheapest cab. It first filters infeasible options, evaluates the remaining alternatives using the customer's preferences, obtains confirmation, and handles changing availability. Consequently, it is a goal-based problem-solving agent with utility-based selection and model-based state tracking.

**QUESTION 5**

A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route has an associated travel cost (fuel + time). Determine the least-cost path from starting warehouse S to destination warehouse G using Uniform Cost Search (UCS).

**Problem understanding and graph representation**

The weighted graph in the question is interpreted using the directed routes shown in the figure:

| Route | Cost | Route | Cost |
|---|---|---|---|
| S â†’ A | 1 | S â†’ G | 12 |
| A â†’ B | 3 | A â†’ C | 1 |
| B â†’ D | 3 | C â†’ D | 1 |
| C â†’ G | 2 | D â†’ G | 3 |

Initial state: S

Goal state: G

Successor function: follow each outgoing route and add its edge cost.

Path cost: sum of all route costs on the path.

UCS rule: always expand the frontier node with the smallest cumulative path cost g(n).

Uniform Cost Search trace

| Step | Node expanded | Frontier after expansion (node : cumulative cost) | Explanation |
|---|---|---|---|
| 0 | â€” | S : 0 | Start with the source. |
| 1 | S : 0 | A : 1, G : 12 | Generate A through Sâ†’A and the direct route Sâ†’G. |
| 2 | A : 1 | C : 2, B : 4, G : 12 | Aâ†’C gives 1+1=2; Aâ†’B gives 1+3=4. |

3 C : 2 D : 3, G : 4, B : 4, G : 12 Câ†’D gives 2+1=3; Câ†’G gives 2+2=4.

4 D : 3 G : 4, B : 4, G : 6, G : 12 Dâ†’G gives 3+3=6, but the existing G : 4 is cheaper.

5 G : 4 â€” G is removed from the priority queue with the lowest cost, so UCS terminates.

Candidate path-cost verification

| Path | Calculation | Total cost |
|---|---|---|
| S â†’ A â†’ C â†’ G | 1 + 1 + 2 | 4 |
| S â†’ A â†’ C â†’ D â†’ G | 1 + 1 + 1 + 3 | 6 |
| S â†’ A â†’ B â†’ D â†’ G | 1 + 3 + 3 + 3 | 10 |
| S â†’ G12 | 12 | |

**Final answer: Uniform Cost Search returns the least-cost path S â†’ A â†’ C â†’ G. The minimum total delivery cost is 4 cost units (the combined fuel-and-time cost represented by the edge weights).**

**Correctness and engineering interpretation**

All route costs are non-negative, so UCS is complete and optimal. The algorithm terminates when G is selected for expansion at cost 4, not merely when G is first generated. This distinction guarantees optimality: every other frontier path has cost at least 4. In engineering terms, the company should use the route Sâ†’Aâ†’Câ†’G because it minimizes the stated combined travel cost and is 8 cost units cheaper than the direct route Sâ†’G.

**End of Answer Script**

LEVEL 4 (EXEMPLARY) RUBRIC ALIGNMENT

The following criteria are explicitly demonstrated in each answer.

**Question 1 - Water Jug Problem**

**Problem Understanding: Capacities, initial state, legal operations and exact goal are defined.**

Method Selection: State-space representation is used because every operation produces a measurable state transition.

Solution Process: Every operation is shown in a numbered state table with no hidden steps.

Accuracy of Calculation: Capacity limits and the final calculation 4 - 2 = 2 gallons are verified.

**Interpretation: The final state is explained using only the permitted operations and no external measuring device.**

**Question 2 - Mars Rover Agent**

**Problem Understanding: Percepts, actions, environment, goals, resources and safety requirements are identified.**

Method Selection: A model-based, utility-based, goal-oriented learning architecture is justified against simpler alternatives.

Solution Process: Percepts, environment properties, actions, performance measures and architecture are organized systematically.

Accuracy of Analysis: The environment classification and utility formulation are consistent with Mars exploration constraints.

**Interpretation: The design is connected to scientific value, risk, energy, communication and reliability.**

**Question 3 - 8-Queens Problem**

**Problem Understanding: State, initial state, actions, transition model, goal test, path cost and constraints are formally specified.**

Method Selection: DFS with backtracking and constraint checking is selected and compared with BFS.

Solution Process: Search-space size, pruning logic, pseudocode and a valid configuration are presented sequentially.

Accuracy of Calculation: 8^8 = 16,777,216 and 8! = 40,320 are correctly calculated and diagonal constraints are verified.

**Interpretation: The configuration is explained as a valid goal state and the strategy is evaluated for memory and completeness.**

**Question 4 - OLA Cab Booking Agent**

**Problem Understanding: Locations, preferences, availability, route, fare, ETA and booking status are identified.**

Method Selection: A goal-based agent with utility-based selection and model-based state tracking is justified.

Solution Process: The pseudocode validates inputs, filters cabs, calculates utility, confirms the choice and handles failures.

Accuracy of Analysis: The utility function correctly rewards comfort and safety and penalizes fare, waiting time and travel time.

**Interpretation: The method is linked to customer priorities, changing availability and safe trip completion.**

**Question 5 - Uniform Cost Search**

**Problem Understanding: Source, goal, weighted routes and cumulative path-cost requirement are explicitly identified.**

Method Selection: UCS is selected because edge costs are non-negative and minimum cumulative cost is required.

Solution Process: The priority-queue trace records every expansion and explains duplicate-cost handling.

Accuracy of Calculation: Candidate path costs 4, 6, 10 and 12 are recalculated; the minimum is unambiguously 4.

**Interpretation: The optimal route is translated into an engineering decision and the 8-unit saving is stated.**